Milestone E

Updated C4 Component Diagram | Game State Machine | State Pattern | Implementation

Embedded Systems - Face Detection Game Project

SDU - Software Engineering

May 2026

1. Updated C4 Component Diagram

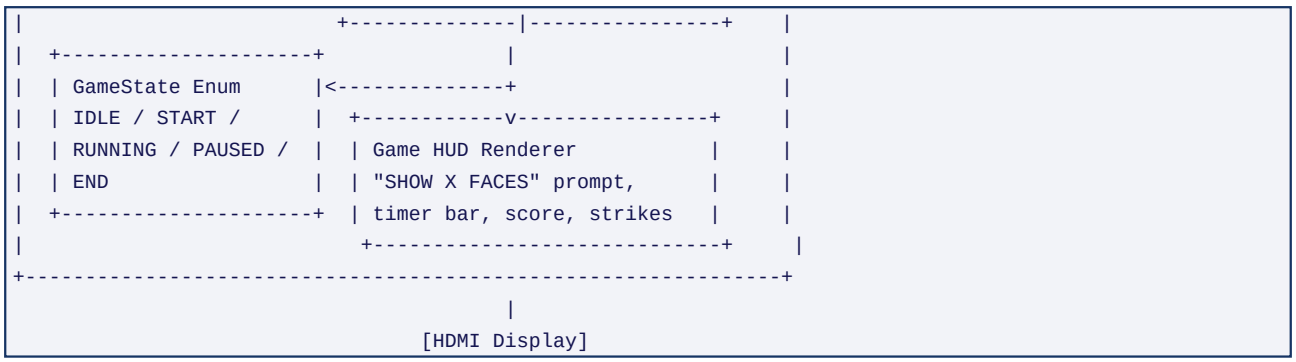
The diagram extends Milestone D by adding the game components: the state machine, HardwareController (GPIO), and the updated HUD renderer that now shows the face target prompt.

1.1 PlantUML Source

```
@startuml Milestone_E_Component
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml
LAYOUT_WITH_LEGEND()
title C4 Component Diagram - Face Detection Game (Milestone E)

Person(player, "Player", "Uses hardware buttons or keyboard")
System_Ext(camera, "Camera", "USB webcam or Pi Camera Module")
System_Ext(display, "HDMI Display", "Renders game output")
System_Ext(gpio, "GPIO Hardware", "4 push-buttons + 2 LEDs on Raspberry Pi")

Container_Boundary(app, "Face Detection Game (face_detector.py + hardware_controller.py)") {
  Component(fd, "FaceDetector", "Python Class",
    "Multi-backend face detection (MediaPipe/DNN/Haar).")
  Component(cd, "CameraFaceDetector", "Python Class / Game Controller",
    "Owns the state machine. Manages score, strikes, round timing.\nRegisters button callbacks w...
  Component(sm, "Game State Machine", "GameState Enum + Logic",
    "Five states: IDLE, START, RUNNING, PAUSED, END.\nTransitions driven by button events or gam...
  Component(hud, "Game HUD Renderer", "Module-level Functions",
    "_draw_game_hud(): score, strikes, face-target prompt, timer.\n_draw_game_over(): end-of-gam...
  Component(hw, "HardwareController", "Python Class",
    "GPIO abstraction. Daemon thread per button with 50ms debounce.\nFalls back to simulation on...
}

Rel(player, gpio, "Presses buttons")
Rel(gpio,   hw,   "Low signal on input pin", "GPIO BCM")
Rel(hw,     cd,   "Fires callback (non-blocking)")
Rel(cd,     sm,   "Drives transitions via start_game/toggle_pause/end_game")
Rel(cd,     fd,   "Sends frame for detection each loop iteration")
Rel(cd,     hud,  "Supplies score, strikes, face count, timer")
Rel(hw,     gpio, "Sets LED pins HIGH/LOW", "GPIO BCM")
Rel(hud,    display, "Renders to", "cv2.imshow / HDMI")
Rel(camera, cd,   "Provides frames", "cv2.VideoCapture")
@enduml
```

1.2 ASCII Overview

```
[Player] --button--> [GPIO Hardware]
                          |
                    [HardwareController]   daemon threads, debounce
                          | callback
                          v
+-------------------------------------------------------------+
|            Face Detection Game Application                   |
|  +-----------------+     +-------------------------------+    |
|  | FaceDetector    |<--->| CameraFaceDetector            |    |
|  | MediaPipe/DNN/  |     | Game controller: owns state   |    |
|  | Haar backends   |     | machine, score, timer, max_   |    |
|  +-----------------+     | faces logic                   |    |
```


GameState Enum
IDLE / START /
RUNNING / PAUSED /
END
Game HUD Renderer
"SHOW X FACES" prompt,
timer bar, score, strikes
[HDMI Display]

2. Functional Game State Machine Model

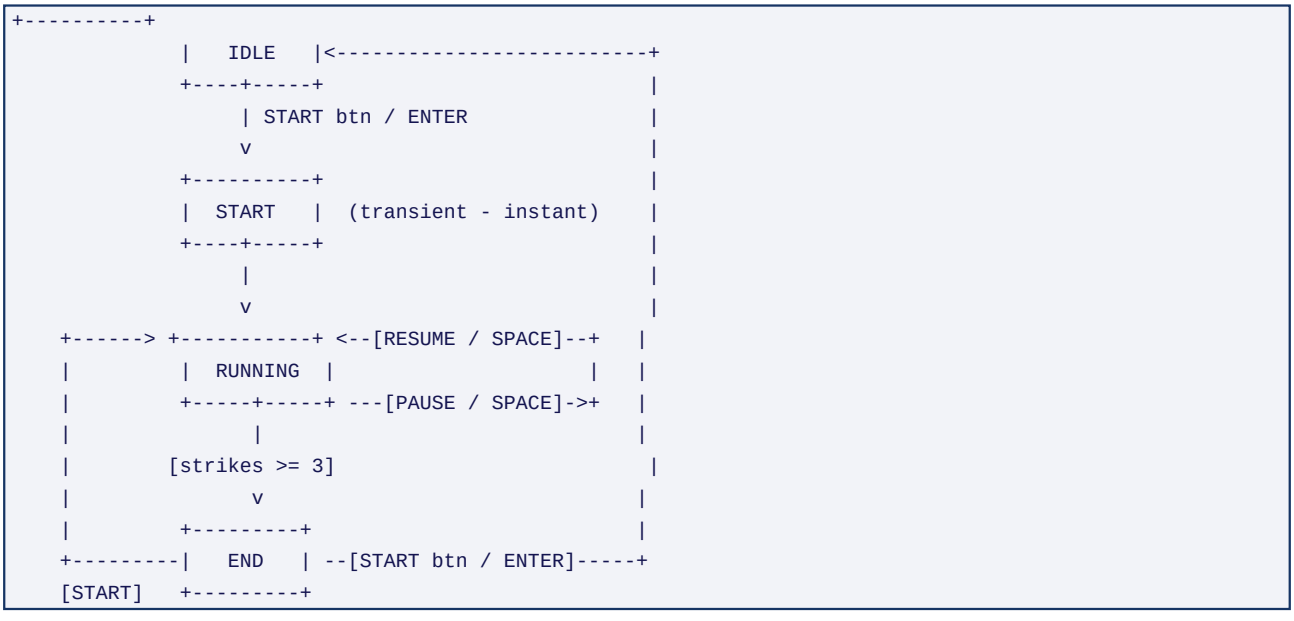
2.1 States

State	Description	Camera active?	Game logic runs?
IDLE	Waiting for player to start. Live camera shown with sta	Yes	No
START	Transient init state: resets score, strikes, timer, flashe	No	No
RUNNING	Active round. Timer counts down. Faces checked aga	Yes	Yes
PAUSED	Timer frozen. Last frame held. 'PAUSED' overlay sho	No	No
END	Game over. Live camera shown with final score overla	Yes	No

2.2 Transitions

From	To	Trigger	Action
IDLE	RUNNING	START button / ENTER key	Reset all state, flash green LED x2
END	RUNNING	START button / ENTER / R key	Reset all state, flash green LED x2
RUNNING	PAUSED	PAUSE button / SPACE key	Record pause_start_time
PAUSED	RUNNING	PAUSE button / SPACE key	Add paused duration to switch_start_time
RUNNING	END	strikes >= 3 (auto)	Set game_over=True, flash red LED x3

2.3 State Diagram (ASCII)



2.4 Timing and Round Logic

Each round lasts get_switch_time() seconds, calculated as max(0.5, 3.0 - score*0.1). The elapsed time is measured with time.perf_counter() and respects pause intervals: when the game is paused the switch_start_time is shifted forward by the paused duration so that elapsed time appears frozen during the

pause.

```
def get_switch_time(self) -> float:
    return max(0.5, self.base_switch_time - self.score * 0.1)

def get_switch_elapsed_time(self) -> float:
    if self.paused:
        return (self.pause_start_time - self.switch_start_time) - self.paused_time_accumulated
    return (time.perf_counter() - self.switch_start_time) - self.paused_time_accumulated
```

3. State Pattern - Concept and Implementation

3.1 Concept

The State pattern (GoF Behavioural Pattern) allows an object to change its behaviour when its internal state changes. The object appears to change its class. In a classic OOP implementation each state is a separate class with a common interface. In embedded Python a lightweight alternative is equally effective: a single enum (GameState) combined with explicit conditional dispatch. This avoids class proliferation while preserving the same structural benefits: every state-dependent decision is gated through a single enum value, making state transitions explicit, testable, and traceable.

3.2 How It Is Applied in This Project

- GameState (Enum) - the state object: defines every valid state as a typed constant.
- CameraFaceDetector - the context: holds self.state and exposes transition methods.
- start_game(), toggle_pause(), end_game() - state transition methods: each updates self.state and performs entry/exit actions (LED feedback, timer reset).
- The game loop - the state-dependent behaviour: uses if/elif on self.state to determine which game logic and HUD overlay to execute each frame.

3.3 State Enum Definition

```
# monitoring_server.py - imported by face_detector.py
class GameState(Enum):
    IDLE    = "idle"     # Waiting for player to press Start
    START   = "start"    # Transient init state (instant)
    RUNNING = "running"  # Active round - timer counting down
    PAUSED  = "paused"   # Timer frozen, last frame held
    END     = "end"      # Game over - final score displayed
```

3.4 Context Class - Transition Methods (with comments)

```
# CameraFaceDetector acts as the STATE CONTEXT
# Each method below is a STATE TRANSITION

def start_game(self) -> None:
    # ENTRY ACTION: reset all game variables, provide LED feedback
    self.state = GameState.START          # transitional state
    self.score = 0
    self.strikes = 0
    self.faces_detected_this_switch = False
    self.reset_switch_timer()
    self.hardware.flash_led(LEDColor.GREEN, duration=0.2, count=2)
    self.state = GameState.RUNNING        # final active state

def toggle_pause(self) -> None:
    # Guards ensure this only fires from valid states
    if self.state not in (GameState.RUNNING, GameState.PAUSED):
        return
    if self.paused:
```

```
        # RESUME: shift the switch timer forward to exclude paused time
        pause_duration = time.perf_counter() - self.pause_start_time
        self.switch_start_time += pause_duration
        self.paused = False
        self.state = GameState.RUNNING
    else:
        # PAUSE: snapshot the current wall-clock time
        self.pause_start_time = time.perf_counter()
        self.paused = True
        self.state = GameState.PAUSED

def end_game(self) -> None:
    # Called automatically when strikes reach 3
    self.state = GameState.END
    self.game_over = True
    self.hardware.flash_led(LEDColor.RED, duration=0.5, count=3)
```

3.5 State-Dependent Behaviour in the Game Loop

```
# The game loop dispatches on self.state each frame --
# this IS the state-pattern behaviour method.

if self.state == GameState.RUNNING:
    if not self.paused:
        # scoring logic - only runs in RUNNING state
        if n_faces >= self.max_faces:
            self.faces_detected_this_switch = True
        if elapsed >= current_switch_time:
            if self.faces_detected_this_switch:
                self.add_score()   # green LED + score++
            else:
                self.add_strike()  # red LED + strike++ -> may call end_game()
            self.faces_detected_this_switch = False
            self.reset_switch_timer()

    if self.state == GameState.RUNNING:   # still running after scoring?
        _draw_game_hud(display, self.score, self.strikes, ...)
        if self.paused:
            _draw_paused_overlay(display)
    else:
        _draw_game_over(display, self.score)  # transition happened this frame

elif self.state == GameState.IDLE:
    _draw_idle_prompt(display)            # "PRESS ENTER TO BEGIN"

elif self.state == GameState.END:
    _draw_game_over(display, self.score)  # final score overlay
```

3.6 Button-to-State Wiring

```
# Hardware buttons registered once at startup:
def _register_button_handlers(self) -> None:
    self.hardware.register_button_callback("button_start", self._on_button_start)
    self.hardware.register_button_callback("button_pause", self._on_button_pause)

# Each handler guards on current state before calling a transition:
def _on_button_start(self, event: ButtonEvent) -> None:
```

```
    if self.state in (GameState.IDLE, GameState.END):
        self.start_game()   # only valid from IDLE or END

def _on_button_pause(self, event: ButtonEvent) -> None:
    if self.state in (GameState.RUNNING, GameState.PAUSED):
        self.toggle_pause() # only valid from RUNNING or PAUSED
```